

Digital or Binary Comparators are made up from standard AND, NOR and NOT gates that compare the digital signals present at their input terminals and produce an output depending upon the condition of those inputs.

For example, along with being able to add and subtract binary numbers we need to be able to compare them and determine whether the value of input A is greater than, smaller than or equal to the value at input B etc. The digital comparator accomplishes this using several logic gates that operate on the principles of *Boolean Algebra*.

There are two main types of **Digital Comparator** available and these are.

- 1. Identity Comparator – an *Identity Comparator* is a digital comparator with only one output terminal for when $A = B$, either $A = B = 1$ (HIGH) or $A = B = 0$ (LOW)
- 2. Magnitude Comparator – a *Magnitude Comparator* is a digital comparator which has three output terminals, one each for equality, $A = B$ greater than, $A > B$ and less than $A < B$

The purpose of a **Digital Comparator** is to compare a set of variables or unknown numbers, for example A ($A_1, A_2, A_3, \dots A_n$, etc) against that of a constant or unknown value such as B ($B_1, B_2, B_3, \dots B_n$, etc) and produce an output condition or flag depending upon the result of the comparison. For example, a magnitude comparator of two 1-bits, (A and B) inputs would produce the following three output conditions when compared to each other.

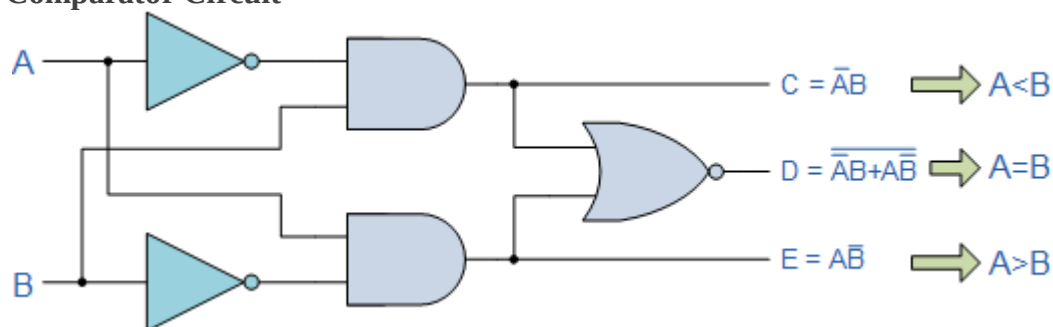
$$A > B, \quad A = B, \quad A < B$$

Which means: A is greater than B, A is equal to B, or A is less than B

This is useful if we want to compare two variables and want to produce an output when any of the above three conditions are achieved. For example, produce an output from a counter when a certain count number is reached.

Consider the simple 1-bit comparator below.

1-bit Digital Comparator Circuit



Then the operation of a 1-bit digital comparator is given in the following Truth Table.

Digital Comparator Truth Table

Inputs		Outputs		
B	A	$A > B$	$A = B$	$A < B$
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0

You may notice two distinct features about the comparator from the above truth table. Firstly, the circuit does not distinguish between either two “0” or two “1”’s as an output $A = B$ is produced when they are both equal, either $A = B = “0”$ or $A = B = “1”$. Secondly, the output condition for $A = B$ resembles that of a

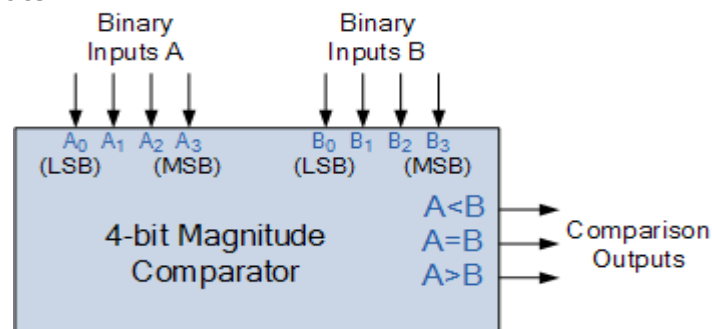
commonly available logic gate, the Exclusive-NOR or Ex-NOR function (equivalence) on each of the n-bits giving: $Q = A \oplus B$

Digital comparators actually use Exclusive-NOR gates within their design for comparing their respective pairs of bits. When we are comparing two binary or BCD values or variables against each other, we are comparing the “magnitude” of these values, a logic “0” against a logic “1” which is where the term **Magnitude Comparator** comes from.

As well as comparing individual bits, we can design larger bit comparators by cascading together n of these and produce a n-bit comparator just as we did for the n-bit adder in the previous tutorial. Multi-bit comparators can be constructed to compare whole binary or BCD words to produce an output if one word is larger, equal to or less than the other.

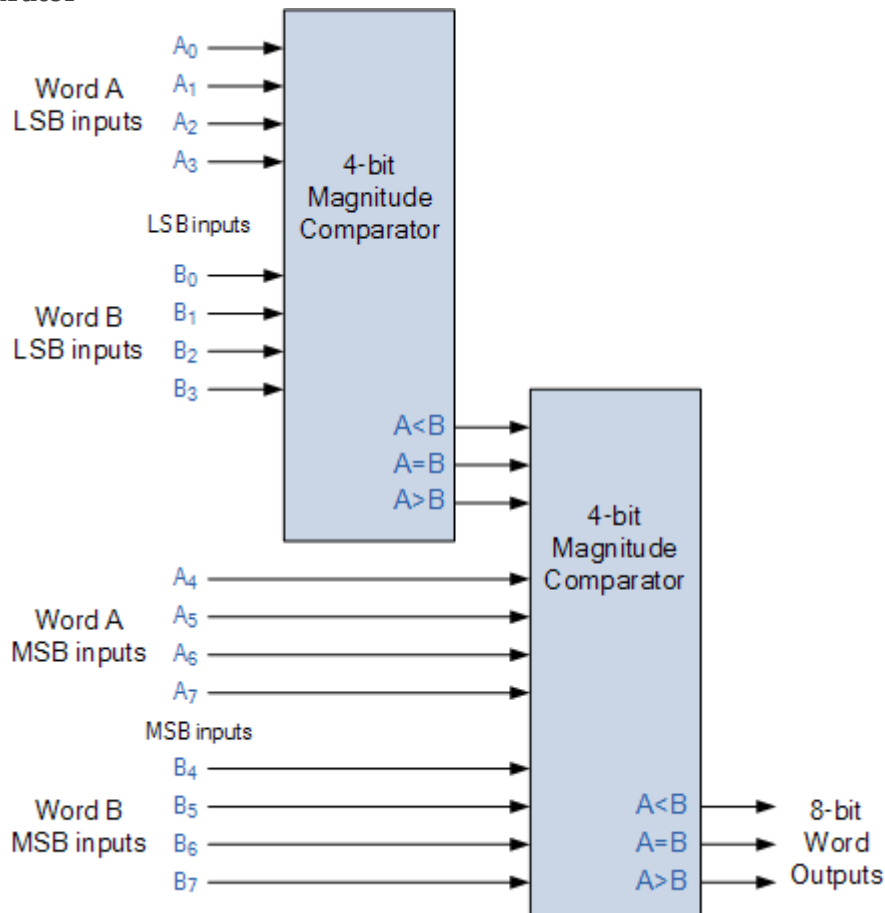
A very good example of this is the 4-bit **Magnitude Comparator**. Here, two 4-bit words (“nibbles”) are compared to each other to produce the relevant output with one word connected to inputs A and the other to be compared against connected to input B as shown below.

4-bit Magnitude Comparator



Some commercially available digital comparators such as the TTL 74LS85 or CMOS 4063 4-bit magnitude comparator have additional input terminals that allow more individual comparators to be “cascaded” together to compare words larger than 4-bits with magnitude comparators of “n”-bits being produced. These cascading inputs are connected directly to the corresponding outputs of the previous comparator as shown to compare 8, 16 or even 32-bit words.

8-bit Word Comparator



When comparing large binary or BCD numbers like the example above, to save time the comparator starts by comparing the highest-order bit (MSB) first. If equality exists, $A = B$ then it compares the next lowest bit and so on until it reaches the lowest-order bit, (LSB). If equality still exists then the two numbers are defined as being equal.

If inequality is found, either $A > B$ or $A < B$ the relationship between the two numbers is determined and the comparison between any additional lower order bits stops. **Digital Comparator** are used widely in Analogue-to-Digital converters, (ADC) and Arithmetic Logic Units, (ALU) to perform a variety of arithmetic operations.

Decimal or BCD Adder

The BCD-Adder is used in the computers and the calculators that perform arithmetic operation directly in the decimal number system. The BCD-Adder accepts the binary-coded form of decimal numbers. The Decimal-Adder requires a minimum of nine inputs and five outputs.

There is the following table used in designing of BCD-Adder.

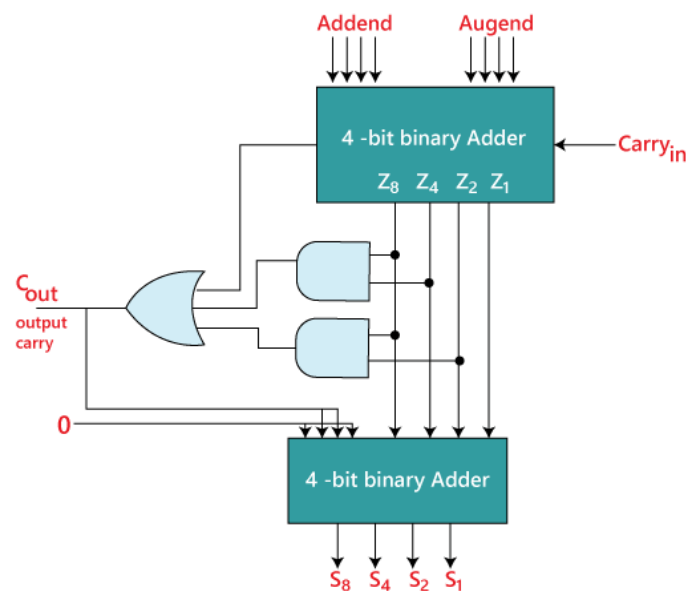
Binary Sum						BCD Sum						Decimal
K	Z ₈	Z ₄	Z ₂	Z ₁		C	S ₈	S ₄	S ₂	S ₁		
0	0	0	0	0		S A M E C O D E	0	0	0	0		0
0	0	0	0	1	0		0	0	0	1		1
0	0	0	1	0	0		0	0	1	0		2
.	.	.	.	.	.		.	.	.	.		.
.	.	.	.	.	.		.	.	.	.		.
.	.	.	.	.	.		.	.	.	.		.
0	1	0	0	0		0	1	0	0	0		8
0	1	0	0	1		0	1	0	0	1		9
10 to 19 Binary and BCD codes are not the same												
0	1	0	1	0		1	0	0	0	0		10
0	1	0	1	1		1	0	0	0	1		11
0	1	1	0	0		1	0	0	1	0		12
0	1	1	0	1		1	0	0	1	1		13
0	1	1	1	0		1	0	1	0	0		14
0	1	1	1	1		1	0	1	0	1		15
1	0	0	0	0		1	0	1	1	0		16
1	0	0	0	1		1	0	1	1	1		17
1	0	0	1	0		1	1	0	0	0		18
1	0	0	1	1		1	1	0	0	1		19

From the above table, it is clear that if the produced sum is between 1 to 9, the Binary and the BCD code is the same. But for 10 to 19 decimal numbers, both the codes are different. In the above table, the binary sum combinations from 10 to 19 give invalid BCD. There are the following points that help the circuit to identify the invalid BCD.

1. It is obvious from the table that a correction is needed when the '**Binary Sum**' has an output carry $K=1$.
2. The other six combinations from 10 to 15 need correction in which the bit on the Z_8 position is 1.
3. In the Binary sum of 8 and 9, the bit on the Z_8 position is also 1. So, the second step fails, and we need to modify it.
4. To distinguish these two numbers, we specify that the bit on the Z_4 or Z_2 position also needs to be 1 with the bit of Z_8
5. The condition for a correction and an output carry can be expressed by the Boolean function:

$$C = K + Z_8 \cdot Z_4 + Z_8 \cdot Z_2$$

Once the circuit found the invalid BCD, the circuit adds the binary number of 6 into the invalid BCD code to make it valid.



Block diagram of a BCD adder

In the above diagram,

1. We take a 4-bit Binary-Adder, which takes addend and augend bits as an input with an input carry '**Carry in**'.
2. The Binary-Adder produces five outputs, i.e., Z₈, Z₄, Z₂, Z₁, and an output carry K.
3. With the help of the output carry K and Z₈, Z₄, Z₂, Z₁ outputs, the logical circuit is designed to identify the C_{out}
4. The Z₈, Z₄, Z₂, and Z₁ outputs of the binary adder are passed into the 2nd 4-bit binary adder as an Augend.
5. The addend bit of the 2nd 4-bit binary adder is designed in such a way that the 1st and the 4th bit of the addend number are 0 and the 2nd and the 3rd bit are the same as C_{out}. When the value of C_{out} is 0, the addend number will be 0000, which produce the same result as the 1st 4-bit binary number. But when the value of the C_{out} is 1, the addend bit will be 0110, i.e., 6, which adds with the augend to get the valid BCD number.

Example: 1001+1000

1. First, add both the numbers using a 4-bit binary adder and pass the input carry to 0.
2. The binary adder produced the result 0001 and carried output 'K' 1.
3. Then, find the C_{out} value to identify that the produced BCD is invalid or valid using the expression

$$C_{out} = K + Z_8 \cdot Z_4 + Z_8 \cdot Z_2$$

$$K = 1$$

$$Z_8 = 0$$

$$Z_4 = 0$$

$$Z_2 = 0$$

$$C_{out} = 1 + 0 \cdot 0 + 0 \cdot 0$$

$$C_{out} = 1 + 0 + 0$$

$$C_{out} = 1$$
4. The value of C_{out} is 1, which expresses that the produced BCD code is invalid. Then, add the output of the 1st 4-bit binary adder with 0110.

$$= 0001 + 0110$$

$$= 0111$$
5. The BCD is represented by the carry output as:

$$BCD = C_{out} Z_8 Z_4 Z_2 Z_1 = 1 0 1 1 1$$

Arithmetic-Logic Unit (ALU)

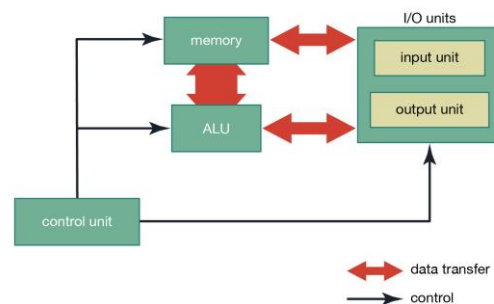
An arithmetic-logic unit (ALU) is the part of a computer [processor \(CPU\)](#) that carries out arithmetic and logic operations on the [operands](#) in computer [instruction words](#). In some processors, the ALU is divided into two units, an arithmetic unit (AU) and a logic unit (LU). Some processors contain more than one AU - for example, one for *fixed-point* operations and another for *floating-point* operations. (In personal computers

floating point operations are sometimes done by a [floating point unit](#) on a separate chip called a numeric coprocessor.)

Typically, the ALU has direct input and output access to the processor controller, main memory (random access memory or [RAM](#) in a personal computer), and input/output devices. Inputs and outputs flow along an electronic path that is called a [bus](#). The input consists of an instruction word (sometimes called a machine instruction word) that contains an operation code (sometimes called an "op code"), one or more operands, and sometimes a format code. The operation code tells the ALU what operation to perform and the operands are used in the operation. (For example, two operands might be added together or compared logically.) The format may be combined with the op code and tells, for example, whether this is a fixed-point or a floating-point instruction. The output consists of a result that is placed in a storage *register* and settings that indicate whether the operation was performed successfully. (If it isn't, some sort of status will be stored in a permanent place that is sometimes called the machine status word.)

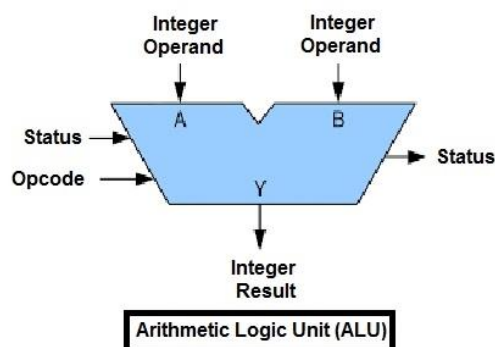
In general, the ALU includes storage places for input operands, operands that are being added, the accumulated result (stored in an *accumulator*), and shifted results. The flow of bits and the operations performed on them in the subunits of the ALU is controlled by gated circuits. The gates in these circuits are controlled by a sequence logic unit that uses a particular [algorithm](#) or sequence for each operation code. In the arithmetic unit, multiplication and division are done by a series of adding or subtracting and shifting operations. There are several ways to represent negative numbers. In the logic unit, one of 16 possible logic operations can be performed - such as comparing two operands and identifying where bits don't match.

The design of the ALU is obviously a critical part of the processor and new approaches to speeding up instruction handling are continually being developed.



© Encyclopædia Britannica, Inc.

Inside a computer, there is an Arithmetic Logic Unit (ALU), which is capable of performing logical operations (e.g. AND, OR, Ex-OR, Invert etc.) in addition to the arithmetic operations (e.g. Addition, Subtraction etc.). The control unit supplies the data required by the ALU from memory, or from input devices, and directs the ALU to perform a specific operation based on the instruction fetched from the memory. ALU is the “calculator” portion of the computer.



An arithmetic logic unit (ALU) is a major component of the central processing unit of a computer system. It does all processes related to arithmetic and logic operations that need to be done on instruction words. In some microprocessor architectures, the ALU is divided into the arithmetic unit (AU) and the logic unit (LU). An ALU can be designed by engineers to calculate many different operations. When the operations become more and more complex, then the ALU will also become more and more expensive and also takes up more space in the CPU and dissipates more heat. That is why engineers make the ALU powerful enough to ensure that the CPU is also powerful and fast, but not so complex as to become prohibitive in terms of cost and other disadvantages.

ALU is also known as an Integer Unit (IU). The arithmetic logic unit is that part of the CPU that handles all the calculations the CPU may need. Most of these operations are logical in nature. Depending on how the ALU is designed, it can make the CPU more powerful, but it also consumes more energy and creates more heat. Therefore, there must be a balance between how powerful and complex the ALU is and how expensive the whole unit becomes. This is why faster CPUs are more expensive, consume more power and dissipate more heat.

Different operations carried out by ALU can be categorized as follows –

- **logical operations** – These include operations like AND, OR, NOT, XOR, NOR, NAND, etc.
- **Bit-Shifting Operations** – This pertains to shifting the positions of the bits by a certain number of places either towards the right or left, which is considered a multiplication or division operation.
- **Arithmetic operations** – This refers to bit addition and subtraction. Although multiplication and division are sometimes used, these operations are more expensive to make. Multiplication and subtraction can also be done by repetitive additions and subtractions respectively.

ALU (Arithmetic Logic Unit)

This unit consists of two subsections namely,

- Arithmetic section
- LogicSection

Arithmetic section

The function of the arithmetic section is to perform arithmetic operations like addition, subtraction, multiplication, and division. All complex operations are done by making repetitive use of the above operations.

Logic Section

The function of the logic section is to perform logic operations such as comparing, selecting, matching, and merging of data.